Laboratorio 8

Introducción - El problema de las N reinas.

El problema de las N reinas es un clásico desafío de lógica y programación que consiste en colocar N reinas en un tablero de ajedrez de N × N sin que ninguna de ellas se amenace entre sí.

Reglas básicas

• En el ajedrez, una reina ataca a cualquier otra pieza que se encuentre en su misma fila, columna o diagonal.

• Para que ninguna se ataque, no puede haber dos reinas compartiendo la misma fila, la misma columna ni la misma diagonal.

• Para un tablero tradicional de 8 × 8 (N = 8), existen 92 soluciones en total, de las cuales 12 son esencialmente distintas (las demás se obtienen mediante rotaciones o reflejos).

• Para N = 2 y N = 3, el problema no tiene solución.

Método de resolución

• Backtracking (Vuelta atrás): Es la técnica más común y eficiente para resolverlo. Consiste en colocar las reinas fila por fila (o columna por columna). Si se detecta que una reina entra en conflicto con las anteriores, el algoritmo "retrocede" (backtrack) y prueba otra posición para la reina anterior.

Tablero de 4x4

0	0	3	0
1	0	0	0
0	0	0	4
0	2	0	0

Definición del problema y requisitos de la solución

El objetivo es implementar en Python un algoritmo que permita encontrar solución (si es que la hay) , para un tablero de NxN.
Se pide:

1. Implementar el algoritmo que permita saber si existe o no solución cuando N = 4, 5, 6, 7, 8. Debe guardar las soluciones que encuentre para cada N en un archivo (solucion.txt). (50 % nota)
2. Implementar el algoritmo que permita encontrar todas las soluciones (si existen) para cada N(Todas_las_soluciones.txt). (30 % nota)
3. Implementar el algoritmo que permita encontrar la mejor solución (si existe) al problema de las N reinas (reinas.txt). (20 % nota).

Plantilla Backtracking iterativo - una solución

```
begin
        #definición particular de cada problema
        while(tenga candidatos AND not solución)do
                if(valido movimiento elegido)then
                        avanzar en la búsqueda de la solución
                        if(llegó al final) then
                                solucion = true
                        else
                                dejar pistas para devolverme
                        endif
                else
                        siguiente candidato
                        while(no estoy en el inicio AND ya no tengo candidatos)do
                                retrocedo
                        endwhile
                endif
        endwhile
end
```

Instrucciones

1. El laboratorio deberá ser desarrollado de manera **individual** utilizando Python.

2. La solución deberá implementarse en un archivo con extensión **.py**, el cual deberá contener el algoritmo solicitado de acuerdo con los requisitos definidos en el laboratorio.
3. El programa deberá ejecutarse correctamente, sin presentar errores de compilación o ejecución, y cumplir con todas las funcionalidades especificadas para el algoritmo.
4. El código deberá seguir los principios de la programación estructurada, utilizando adecuadamente variables, funciones, estructuras de control y demás elementos vistos durante el curso.
5. Se deberán considerar los casos de prueba necesarios para verificar el correcto funcionamiento del programa, incluyendo la validación de las entradas cuando corresponda.
6. Utilizar como base de trabajo Pseudo código en el ANEXO

Evaluación.

1. La entrega del laboratorio será **individual** y deberá realizarse dentro del plazo establecido por el docente.
2. La evaluación se regirá por la **Pauta de Evaluación de Laboratorio** correspondiente a la asignatura, considerando tanto el correcto funcionamiento del programa como el cumplimiento de los requisitos establecidos.
3. Como parte de la evaluación, el estudiante deberá realizar una **defensa del código desarrollado**, en la cual deberá explicar la lógica de la solución implementada, el funcionamiento del algoritmo y las decisiones tomadas durante su desarrollo.
4. Durante la defensa, el docente podrá realizar preguntas relacionadas con el código presentado y solicitar modificaciones o explicaciones sobre partes específicas del programa, con el fin de verificar la comprensión del estudiante y la autoría del trabajo entregado.
5. Todos los trabajos serán evaluados en el laboratorio
6. **FECHA DE ENTREGA: 18 DE OCTUBRE DE 2026 HASTA LAS 24 HORAS.**

ANEXO

```
#################################################################
BEGIN N – REINAS MÓDULO PRINCIPAL
        # N DEFINE EL TAMAÑO DEL LABERINTO
        TABLERO[N][N]
        CREAR_TABLERO(TABLERO,N)
        IF(BUSCA_SOLUCIÓN(TABLERO,N))THEN
                IMPRIMIR_SOLUCION(TABLERO,N)
        ELSE
                WRITE(“NO HAY SOLUCION”)
        ENDIF
END
#################################################################


#################################################################
BEGIN BUSCA_SOLUCION (TABLERO, N)
        REINA = 1; CANDIDATO = 0; COLUMNA = 0;
        TABLERO(CANDIDATO, COLUMNA) = REINA; SOLUCION = FALSE;
        COLUMNA = COLUMNA +1 ;
        WHILE (CANDIDATO < N AND NOT SOLUCION)DO
                IF(VALIDO(TABLERO, CANDIDATO, COLUMNA))THEN
                        TABLERO(CANDIDATO, COLUMNA)= REINA +1
                        IF(COLUMNA == N-1)THEN
                                SOLUCION = TRUE
                        ELSE
                                COLUMNA = COLUMNA + 1; REINA=REINA+1;
                                CANDIDATO = 0
                        ENDIF
                ELSE
                        CANDIDATO ++;
                        WHILE((COLUMNA == 0) AND CANDIDATO == N)DO
                                REINA = REINA+1;
                                CANDIDATO, COLUMNA = BUSCA_XY(TABLERO, REINA);
                                TABLERO(CANDIDATO, COLUMNA) = 0
                                CANDIDATO = CANDIDATO+1
                        ENDWHILE
```

```
            ENDIF
        ENDWHILE
END
##################################################################

##################################################################
BEGIN VALIDO (TABLERO, CANDIDATO, COLUMNA)
        FOR I=0; I<N; I++
            IF(TABLERO(I,COLUMNA) ¡=0)THEN
                RETURN FALSE
            ENDIF
        ENDFOR
        FOR J=0; J<N; J++
            IF(TABLERO(CANDIDATO,J) ¡=0)THEN
                RETURN FALSE
            ENDIF
        ENDFOR
        ###### DP HACIA ARRIBA
        I = CANDIDATO; J = COLUMNA;
        WHILE(I>=0 AND J>=0)DO
            IF(TABLERO(I,J) ¡=0)THEN
                RETURN FALSE
            ENDIF
            I=I-1;J=J-1
        ENDDO
        ###### DP HACIA ABAJO
        I = CANDIDATO; J = COLUMNA;
        WHILE(I<N AND J<N)DO
            IF(TABLERO(I,J) ¡=0)THEN
                RETURN FALSE
            ENDIF
            I=I+1;J=J+1
        ENDDO
        ###### DS HACIA ARRIBA
        I = CANDIDATO; J = COLUMNA;
        WHILE(I>=0 AND J<N)DO
            IF(TABLERO(I,J) ¡=0)THEN
                RETURN FALSE
```

```
            ENDIF
            I=I-1;J=J+1
        ENDDO
        ###### DS HACIA ABAJO
        I = CANDIDATO; J = COLUMNA;
        WHILE(I<N AND J>=0)DO
            IF(TABLERO(I,J) ¡=0)THEN
                RETURN FALSE
            ENDIF
            I=I+1;J=J-1
        ENDDO
        RETURN TRUE
END
#################################################################


#################################################################
BEGIN BUSCA_XY (TABLERO, VALOR)
        FOR I=0;I<N,I++
            FOR J=0;J<N;J++
                IF(TABLERO(I,J)==VALOR)THEN
                    RETURN I,J
                ENDIF
            ENDFOR
        ENDFOR
END
#################################################################
```